

Ch1

Course Info

- Test (20%)
- Assignments/classwork (15%)
- Quizzes (15%)
- Exam (50%)

Learning Objective

- **Analyze** the architecture of enterprise systems, including multi-tier, and microservices patterns.
- **Design** and implement transactional business logic using enterprise frameworks.
- **Integrate** applications with relational databases using ORM (Object-Relational Mapping) and connection pooling.
- **Build** RESTful APIs for internal and external system communication.

a. Analyze 架构分析：多层架构 & 微服务

一个企业系统是如何被拆分成不同层次或不同服务的。

- **多层架构**：典型三层（表示层 / 业务层 / 数据层）。
- **微服务**：把系统拆成多个独立部署的小服务，每个服务负责单一业务能力。

为什么重要

- 决定系统的可扩展性、可维护性、部署方式。
- 影响你后续所有设计：事务、数据库、API、服务间通信。

b. Design 设计并实现事务型业务逻辑

掌握如何在企业框架中写“可靠的业务流程”。

- 事务：保证一组操作要么全部成功，要么全部失败。
- 企业框架：如 Spring、Jakarta EE、NestJS 等。

为什么重要

- 企业系统的核心是业务流程（下单、支付、审批、库存扣减）。
- 事务处理不当会导致数据不一致、重复扣费、库存错误。

c. Integrate 数据库集成：ORM + 连接池

把应用和数据库正确连接起来，并高效读写数据。

- ORM：把数据库表映射成对象（如 JPA、Hibernate、MyBatis）。
- 连接池：复用数据库连接，提高性能（如 HikariCP）。

为什么重要

- 企业系统大量依赖数据库，性能和稳定性都取决于连接管理。
- ORM 能减少重复 SQL，提高开发效率。

d. Build 构建 RESTful API

设计和实现系统之间通信的接口。

- RESTful API：基于 HTTP 的资源式接口设计。
- 用于前端、内部服务、外部系统之间的通信。

为什么重要

- 企业系统通常是分布式的，API 是所有服务协作的基础。
 - API 质量直接影响系统的可扩展性和可维护性。
-

CH 1 Introduction to Enterprise System (ES)

1. Enterprise Computing

What is the Enterprise Computing?

An organization (company, charity,
Government Department / Agency)

the use of computers to process
data or perform calculations

What is enterprise computing?

- Enterprise computing is about combining separate applications, services, and process into unified system that is greater than the sum of its parts.
- Enterprise computing is usually seen as a collection of big business IT solutions to common problems such as resource management and streamlining processes.
- People study the common problems (business values) and provide IT solutions (product, platform, framework, service, etc.)
- Enterprise computing involves many different roles (manager, architect, administrator, developer, analyst, operator, support, etc.) for different areas (solution, system, database, network, data, quality assurance, etc.).

Which enterprise needs enterprise computing?

Types of enterprises include:

- Retail
- Manufacturing
- Service
- Wholesale
- Government
- Educational
- Transportation
- etc.

2. Enterprise System (ES)

Enterprise System (ES)

- An enterprise system is a comprehensive software application designed to integrate and automate various business processes across an entire organization. It provides a centralized platform for managing data, workflows, and resources, enabling better collaboration, efficiency, and decision-making.
- A software system designed to operate in a corporate environment such as business or government: *targets an organization rather than individual users*
- Typically, is a **Complex, large-scale, mission-critical, portable, scalable, secure, transactional, distributed system.**
- Consists of a group of applications sharing various resources (users, clients, data, etc.) and integrating core business processes: sales, accounting, finance, human resources, inventory and manufacturing, document management, etc.

企业系统是一类 整合多个业务流程、跨部门协作的大型软件系统。

a. Key characteristics

1. High Performance

- Able to provide rapid transaction time even in peak hours.
- Able to process massive amount of transactions concurrently 并发处理

高性能是企业系统的生命线。它意味着：

- **低延迟**：请求响应时间稳定在毫秒级。
- **高吞吐**：每秒处理成千上万次交易（TPS）。
- **可扩展性**：负载上升时能自动扩容（如使用负载均衡、分布式架构）。

2. Stable & Robustness 稳定性与健壮性

- Server running consistently and correctly under any kinds of legal and illegal operations.
- Constantly amend and improve to prevent loopholes. 持续修补和改进以防止漏洞
- System environmental upgrade to fix potential problems.

稳定性与健壮性意味着系统必须：

- **抗异常**：非法输入、恶意请求不会导致系统崩溃。
- **抗故障**：单个组件失败不会影响整体服务（如熔断、降级）。
- **持续维护**：补丁、版本升级、漏洞修复必须常态化。

特性	异常类型	范围	目标
Stable/Robustness	小规模异常（非法输入、单模块错误）	软件层面	不崩、不挂、正确运行
Resilience	灾难级异常（断电、机房挂掉、区域故障）	系统整体层面	持续运行或快速恢复

3. Resilience Control 弹性控制 / 韧性设计

- Anti-virus, malicious software prevention. 防病毒、恶意软件防护
- Power supply shortage. 应对电力供应不足等突发情况。

4. Cost Effectiveness 成本效益

Short development effort but high profitable product. 开发周期短，但产品利润高。

企业系统不仅要强大，还要 **成本可控**：

- 开发周期短 → 使用成熟框架、模块化设计、复用组件
- 维护成本低 → 自动化运维、监控、日志系统

- 投资回报高 → 系统能显著提升效率、减少人工成本

5. Extensible & Flexible 可扩展与灵活性

- Ease of further development and integration with others.

企业系统必须能随着业务增长不断扩展：

- **可扩展性**：能轻松增加新模块、新功能。
- **灵活性**：能与第三方系统集成（API、消息队列、ESB）。
- **可维护性**：代码结构清晰、模块解耦、文档完善。

6. Disaster Recovery (DR) 灾难恢复

- Data & software backup and recovery mechanism.
- Data Centers and DR sites are required.

DR 是企业系统面对“灾难级事件”时的 **恢复能力**，属于系统可靠性最高层级。

它解决的问题是：数据丢，机房损坏，火灾、洪水、地震，整个区域的服务不可用

7. High Availability (HA) 高可用性

- Provide almost zero downtime services (24 × 7).
- Load balancing, failover, virtual servers, etc. 负载均衡、故障切换、虚拟服务器

HA 的目标是：**系统永不停机**。

8. Resource Management & Optimization

Database, application servers, operating system, network optimization.

企业系统规模大，资源消耗高，因此必须进行系统级优化：

- **数据库**：索引、缓存、分库分表、读写分离
- **应用服务器**：线程池、连接池、GC 调优
- **操作系统**：内核参数、文件句柄、IO 调度
- **网络**：带宽、延迟、路由、负载均衡策略

9. Standardization & Documentation 标准化与文档化

- Provide **enough documents** (system handbooks, operational guidelines, administration whitepaper, etc.) to explain the system.
- Acquire ISO to reduce operational risks, maintain high quality of services and secure to achieve industrial standard level. 获得 ISO 等行业标准，以降低运营风险、保持高质量服务并达到行业标准
- Code of Practice for **staffs** to follow the operational procedures.

特性	工程化重点	关注层级
High Performance 高性能	并发、吞吐、缓存、分布式、消息队列	性能层
Stable & Robustness 稳定性与健壮性	异常处理、熔断、限流、重试、输入校验	软件层抗异常
Resilience Control 韧性 / 弹性控制	多活、容灾、UPS、防病毒、混沌工程	系统层抗灾难
High Availability (HA) 高可用性	负载均衡、故障切换、滚动升级、健康检查	架构层持续运行
Disaster Recovery (DR) 灾难恢复	RPO/RTO、异地备份、快照、日志复制、灾备演练	灾备层恢复能力
Resource Optimization 资源管理与优化	DB/OS/网络调优、连接池、索引、内核参数	性能调优层
Cost Effectiveness 成本效益	开源框架、自动化运维、云服务、模块复用	成本控制层
Extensible & Flexible 可扩展与灵活性	微服务、插件式架构、标准化 API、消息中间件	扩展性层
Standardization & Documentation 标准化与文档化	ISO 标准、系统手册、操作规范、审计流程	管理与合规层

b. Popular enterprise systems

- Human Capital Management (HCM)
- Business Intelligence (BI)
- Content Management System (CMS)
- Customer Relationship Management (CRM)
- Enterprise Application Integration (EAI)
- Enterprise Forms Automation
- Supply Chain Management (SCM)
- Enterprise Resource Planning (ERP)
- Key Performance Indicator (KPI)
- Online Shopping and payment processing.

- **Enterprise Resource Planning (ERP):**
 - Integrates **core business processes** such as finance, inventory, manufacturing, and sales.
 - SAP, Oracle, Microsoft Dynamics.
- **Customer Relationship Management (CRM):**
 - Manages **interactions with customers and prospects**, including sales, marketing, and customer service.
 - Salesforce, Microsoft Dynamics, HubSpot.
- **Supply Chain Management (SCM):**
 - **Optimizes the flow of goods and services from suppliers to customers**, including procurement, production, and logistics.
 - Oracle, SAP, JDA Software
- **Human Capital Management (HCM):**
 - Manages **all aspects of the employee lifecycle**, from recruitment and onboarding to performance management and payroll.
 - Workday, Oracle, SAP

c. Evolution of Enterprise System

1. Mainframe Era (1960s–1980s) 主机时代

集中式计算：所有计算都在大型主机上完成

批处理（Batch Processing）：每天或每周跑批任务

用户交互极少：通常是命令行或卡片输入

2. Client–Server Era (1980s–2000s) 客户端–服务器时代

分布式计算：客户端负责 UI，服务器负责逻辑

图形界面（GUI）：用户体验大幅提升

网络化：企业开始内部联网

3. ERP Era (1990s–Present) ERP 时代

业务流程整合：财务、库存、制造、销售全部统一

实时数据：企业第一次能实时看到业务状态

标准化流程：企业流程被软件固化

4. Web–Based Enterprise Systems (2000s–Present) 基于 Web 的企业系统

浏览器访问：不再需要安装客户端

SaaS 模式出现

部署成本下降

5. Mobile & Cloud–Native Era (2010s–Present) 移动与云原生时代

移动化：随时随地访问企业系统

云原生：系统为云而生，而不是迁移到云

微服务架构：系统拆成小服务，独立部署

时代	时间范围	文档原句（引用）	核心特征总结	代表技术 / 系统
Mainframe Era	1960s—1980s	“Focus: Batch processing, centralized computing, and limited user interaction.”	集中式计算、批处理、低交互、昂贵且难维护	IBM Mainframe、COBOL、早期库存/财务系统
Client—Server Era	1980s—2000s	“Distributed computing, GUIs, networking... relational databases.”	分布式计算、图形界面、网络化、关系型数据库	Oracle、SQL Server、SAP R/2、桌面应用
ERP Era	1990s—至今	“Integrated business processes... real-time information... enterprise-wide integration.”	业务流程整合、实时数据、企业级统一平台	SAP R/3、Oracle EBS、Microsoft Dynamics
Web-Based ES	2000s—至今	“Web accessibility... cloud computing... reduced infrastructure costs.”	Web 化访问、SaaS 出现、部署成本下降	Salesforce、Workday、NetSuite
Mobile & Cloud—Native Era	2010s—至今	“Mobile accessibility... cloud-based infrastructure... microservices architecture.”	移动化、云原生、微服务、高扩展性	AWS/Azure/GCP、Kubernetes、Docker、SAP S/4HANA

d. Benefits of enterprise system

- Improved efficiency and productivity 提效
- Enhanced data visibility and reporting 可视化
- Increased collaboration and communication 合作
- Better decision-making based on real-time data 决策
- Reduced costs through automation and process optimization 花销
- Improved customer service and satisfaction 用户满意
- Enhanced compliance with regulations 合规

e. Challenges

- High implementation costs
- Complexity and integration challenges
- Resistance to change from employees
- Potential for data security breaches
- Ongoing maintenance and support requirements

在自动化和流程优化的 cost 减少了，但是实现过程中的 cost 反而增加

3. Distinction between enterprise software and consumer software

1. Target Users & Stakeholders

- **Consumer Software (B2C):** Targets **end-users** (individuals). Users are diverse, with varying technical skill levels. The decision-maker is the user themselves.
- **Enterprise Software (B2B):** Targets **organizations** (employees, partners, admins). Users are often domain experts (accountants, HR, logistics), but the **purchaser** is usually a CIO, CTO, or department head, not the end-user. Requirements often come from business analysts, not the users directly.

2. Core Value Proposition & UX

- **Consumer: Experience-first.** Success depends on delight, ease of use, intuitive interfaces, and emotional engagement. If it's frustrating, users uninstall. Design consistency and aesthetics (e.g., Figma-driven UI) are paramount.
- **Enterprise: Functionality-first.** Success depends on accurately executing complex business rules, automating workflows, and improving efficiency. The UI can be "clunky" or data-dense (dashboards, tables, grids) as long as it gets the job done reliably. Usability is secondary to utility.

3. Non-Functional Requirements

- **Consumer:**

- **Scalability: Horizontal** (handle millions of concurrent users during viral spikes).
- **Availability:** Usually **high (99.9%)**, but users **tolerate brief outages** (e.g., Instagram down for 5 minutes).
- **Consistency:** Often **Eventual Consistency** is acceptable (e.g., seeing a "stale" like count temporarily).

1. Horizontal (handle millions of concurrent users during viral spikes).

- 需要横向扩展（加更多服务器）
- 流量不可预测（例如突然爆红）
- 用户规模可能瞬间从 1 万 → 100 万

2. Usually high (99.9%), but users tolerate brief outages

- 99.9% 可用性已经很高
- 偶尔宕机几分钟用户能接受

3. Eventual Consistency is acceptable

- 最终一致即可
- 用户看到的点赞数可能暂时不准确
- 不影响业务

- **Enterprise:**

- **Scalability: Vertical & Predictable** (scaling based on known user/transaction counts from contracts).
- **Availability: Mission-critical (99.99%+).** Downtime means halted business operations (no payroll, no shipping). Requires 24/7/365 support and HA/DR (High Availability/Disaster Recovery) clusters.
- **Consistency: Strict ACID** (Atomicity, Consistency, Isolation, Durability) is non-negotiable for financial/order data. You cannot "eventually" deduct money from a bank account.

1. Vertical & Predictable (scaling based on known user/transaction counts)

- 用户量可预测（合同里写了多少员工、多少交易）
- 扩展方式通常是垂直扩展（提升单机性能）

2. Mission-critical (99.99%+). Downtime means halted business operations

3. Strict ACID... non-negotiable for financial/order data.

- 必须严格 ACID
- 金融、订单、库存不能出现“最终一致”

- 银行不能“过几秒再扣钱”

4. Security & Compliance

- **Consumer:** Focuses on **data privacy** (GDPR/CCPA), password security, and preventing account takeovers. Authentication is often OAuth2 via social logins (Google/Facebook).
- **Enterprise:** Focuses on **zero-trust, perimeter security, and auditing.**
 - Requires complex **RBAC (Role-Based Access Control)** and **ABAC (Attribute-Based Access Control)**.
 - Mandatory **audit trails** (who did what, when, and why) for legal/regulatory reasons (e.g., HIPAA for healthcare, SOX for finance, PCI-DSS for payments).
 - Often integrates with corporate **LDAP/Active Directory** for Single Sign-On (SSO), not social logins.

1. Security Focus 安全重点

Consumer (消费级软件)

“Focuses on data privacy (GDPR/CCPA), password security, preventing account takeovers.”

- 保护用户隐私
- 防止密码泄露
- 防止账号被盗（如短信验证码、双因素认证）

Enterprise (企业级软件)

“Zero-trust, perimeter security, auditing.”

- 零信任架构（默认不信任任何内部或外部请求）
- 边界安全（防火墙、VPN、WAF）
- 审计与合规

工程化理解

- 所有请求必须认证
- 内网也不默认安全
- 必须记录所有关键操作

2. Authentication 认证方式

Consumer

“Authentication is often OAuth2 via social logins (Google/Facebook).”

- 用户用 Google、Facebook 登录
- 方便、快速、无需企业级权限管理

Enterprise

“Integrates with corporate LDAP/Active Directory for Single Sign-On (SSO).”

- 企业内部统一身份系统
- 员工用公司账号登录所有系统
- 权限由企业 IT 管理

3. Authorization 授权方式

Consumer

- 简单权限：普通用户、管理员
- 不需要复杂的权限模型

Enterprise

“Requires complex RBAC and ABAC.”

- RBAC：基于角色（如 HR、财务、仓库）
- ABAC：基于属性（部门、岗位、地区、项目）
- 权限必须精细到字段级、操作级

工程化理解

- 企业系统必须确保“谁能看什么、谁能改什么”
- 权限错误可能导致严重法律风险

4. Compliance 合规要求

Consumer

“Focuses on data privacy (GDPR/CCPA).”

- 主要是隐私法规
- 用户数据不能乱用

Enterprise

“HIPAA... SOX... PCI-DSS.”

- 医疗系统必须符合 HIPAA

- 金融系统必须符合 SOX
- 支付系统必须符合 PCI-DSS

5. Audit Trails 审计日志

Consumer

- 基本日志即可
- 不需要记录“谁做了什么”

Enterprise

“Mandatory audit trails (who did what, when, and why).”

- 必须记录每个关键操作
- 必须可追溯
- 必须满足法律审计要求

5. Integration & Interoperability (Cont.)

- **Consumer:** Typically **standalone**. Integrates mainly with **standard external APIs** (e.g., Stripe for payments, Google Maps for location). The ecosystem is relatively small.
- **Enterprise: Deeply interconnected.** No enterprise app is an island. It must integrate with:
 - Legacy systems (mainframes, COBOL) via SOAP/XML or file transfers (SFTP).
 - Other enterprise modules (ERP, CRM, SCM).
 - Uses **ESB (Enterprise Service Bus)** or message queues (JMS/Kafka) for asynchronous data synchronization across departments.
 - *Result:* The system becomes a "system of record," not just a "system of engagement."

5. Integration & Interoperability (Cont.)

- **Consumer:** Typically **standalone**. Integrates mainly with **standard external APIs** (e.g., Stripe for payments, Google Maps for location). The ecosystem is relatively small.
- **Enterprise: Deeply interconnected.** No enterprise app is an island. It must integrate with:
 - Legacy systems (mainframes, COBOL) via SOAP/XML or file transfers (SFTP).
 - Other enterprise modules (ERP, CRM, SCM).
 - Uses **ESB (Enterprise Service Bus)** or message queues (JMS/Kafka) for asynchronous data synchronization across departments.
 - **Result:** The system becomes a "system of record," not just a "system of engagement."

维度	Consumer (消费级软件)	Enterprise (企业级软件)	工程化解释
Integration Scope 集成范围	小、轻量、标准 API	大、复杂、跨部门、跨系统	消费级只需对接几个外部服务；企业级必须整合整个组织的系统
Legacy Integration 旧系统集成	几乎没有	必须对接主机、COBOL、老系统	企业系统不能抛弃旧系统，只能兼容它们
Enterprise Modules 企业模块集成	不需要	ERP、CRM、SCM、HCM 等深度互联	企业系统必须跨部门协作
Integration Technology 集成技术	REST API、OAuth	ESB、SOAP/XML、SFTP、JMS、Kafka	企业系统需要可靠、可审计、可追溯的集成方式
Data Sync 数据同步	简单同步	异步同步、跨部门数据流	企业数据必须一致、可追踪
System Role 系统角色	System of Engagement (用户交互)	System of Record (业务数据源头)	企业系统是“数据真相来源”，不能出错

6. Data Volume & Complexity

- **Consumer:** Manages high volumes of **user-generated content** (photos, posts, comments) but with a relatively **simple relational schema** (User, Post, Like).
- **Enterprise:** Manages **massive transactional volumes** (thousands of orders per second) with **highly complex schemas** (hundreds of tables with deep normalization). Batch processing (overnight jobs for billing, payroll, and reporting) is a standard enterprise requirement.

7. Development Lifecycle & Deployment

- **Consumer: Agile & High-velocity.** Features ship daily or weekly. "Move fast and break things."
 - Deployment: **Public Cloud (SaaS)**. Canary releases and A/B testing are standard.
 - Feedback loop: Immediate (app store ratings, crashlytics, user analytics).
- **Enterprise: Structured & Predictable.**
 - Lifecycle: Often follows **Waterfall** or "Scaled Agile" with strict governance. Release cycles can be **months or years** (e.g., annual versions).
 - Deployment: **On-premise, Private Cloud, or Hybrid**. Customers often require the software to run inside their own secure data centers (firewalled).
 - Backward Compatibility: **Sacred**. New versions *must* support old APIs and database schemas to avoid breaking existing client integrations.

8. Maintenance & Support

- **Consumer: Best-effort support.** Relies on **community** forums, FAQ, and automated bots. Bug fixes are deployed via hotfixes within hours.
- **Enterprise: SLA-based (Service Level Agreements).** Offers **24/7 premium support** with guaranteed response times (e.g., P1 critical bug fixed within 4 hours). Patches and maintenance updates are rigorously tested and scheduled during narrow maintenance windows.

9. Cost & Business Model

- **Consumer:** **Low entry barrier.** Freemium, low-cost subscriptions (\$5/month), or ad-supported. Acquisition cost is marketing-driven.
- **Enterprise:** **High cost & long-term contracts.** Licenses are often priced per user/per module (e.g., \$10,000+ annually). Sales involve lengthy procurement processes, RFPs (Request for Proposals), and legal reviews.

Dimension	Consumer Software	Enterprise Software
User	General public	Employees, stakeholders, admins
UX Priority	Delight, simplicity, emotional engagement	Efficiency, data density, workflow accuracy
Transaction	Eventual consistency accepted	Strict ACID compliance (JTA/Transactions)
Security	Privacy, social login	SSO, LDAP, RBAC, Audit logs (JWT/Jakarta Security)
Deployment	100% Public Cloud (SaaS)	On-premise, Hybrid, Private Cloud

Dimension	Consumer Software	Enterprise Software
Release Cycle	Continuous (daily/weekly)	Scheduled (quarterly/yearly) with strict regression testing
Integration	Few external APIs	Massive legacy & 3rd-party integrations (JMS, SOAP, Kafka)
Lifecycle	Short-lived; new version replaces old	Long-lived (10+ years); backward compatibility mandatory
Support	Automated/Community	24/7 with contractual SLAs